

KIỂM THỬ HỘP TRẮNG

Ths. Nguyễn Thu Hường

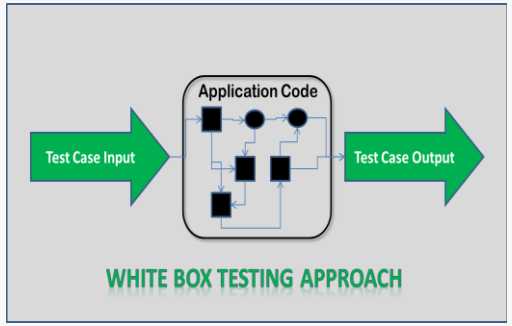
NỘI DUNG

- Khái niệm về kiểm thử hộp trắng
- Kiểm thử luồng điều khiển
- Kiểm thử luồng dữ liệu

KHÁI NIỆM VỀ KIỂM THỬ HỘP TRẮNG

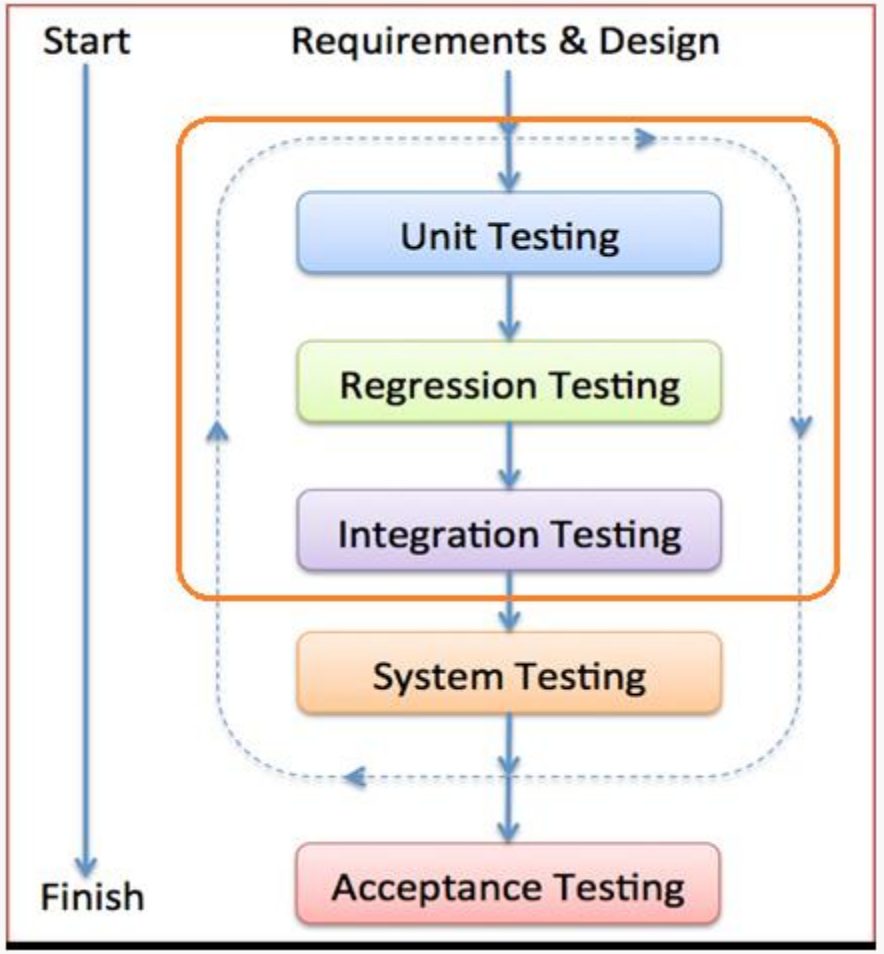
Kiểm thử hộp trắng

- **Kiểm thử hộp trắng – white box testing**, hay còn gọi là kiểm thử dựa vào cấu trúc.
- Là một kỹ thuật kiểm thử dựa trên cấu trúc dữ liệu và thuật toán được cài đặt bên trong của chương trình.
- **Mục tiêu:** xác định hoạt động của phần mềm thông qua việc cài đặt – quá trình validation.
- **Yêu cầu:** người kiểm thử phải có kiến thức về ngôn ngữ lập trình.



Các mức kiểm thử

- Kiểm thử hộp trắng được áp dụng cho: Unit Test, Regression Test.
- Tuy nhiên cũng có thể áp dụng cho Integration Test, System Test.



Kiểm thử hộp trắng

❖ Ưu điểm

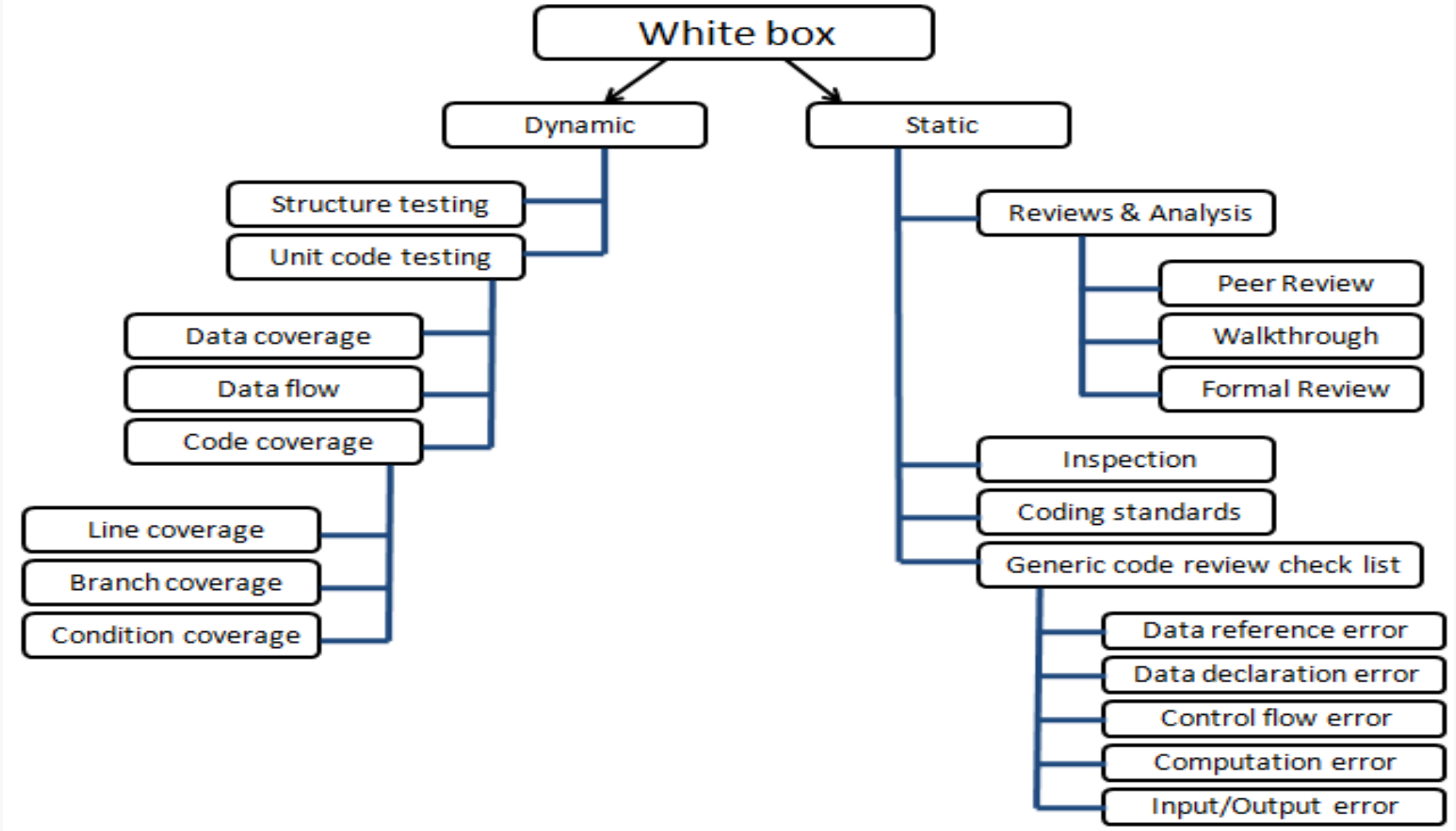
- Việc kiểm thử có thể bắt đầu ở giai đoạn sớm hơn, ngay cả khi giao diện người dùng chưa được xây dựng.
- Kiểm thử kỹ hơn, có thể bao phủ hầu hết các đường dẫn trong mã lệnh.
- Thích hợp trong việc tìm kiếm lỗi và các vấn đề trong mã lệnh.
- Các lập trình viên có thể tự kiểm thử.
- Do yêu cầu kiến thức cấu trúc bên trong của phần mềm, nên việc kiểm soát lỗi tối đa nhất.

Kiểm thử hộp trắng

❖ Nhược điểm

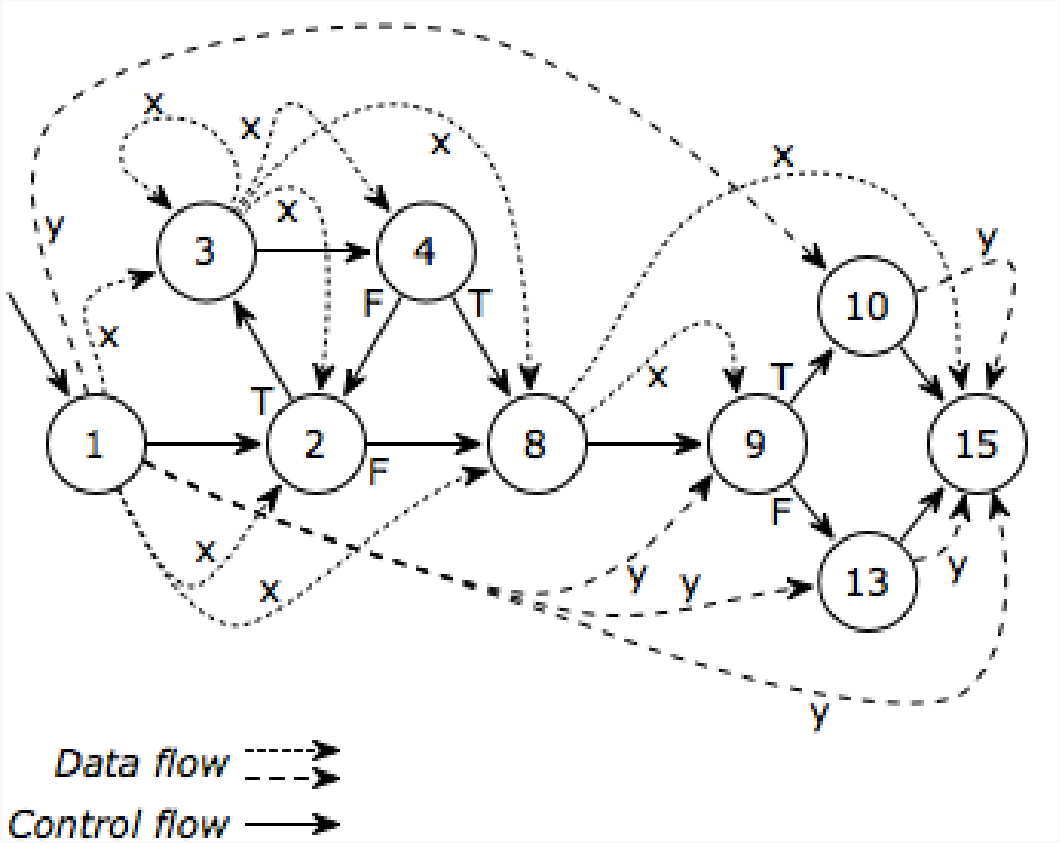
- Đòi hỏi phải có các nguồn lực có tay nghề cao, với kiến thức sâu rộng về lập trình và việc thực hiện các hàm đơn vị.
- Tốn khá nhiều thời gian để kiểm tra được chi tiết cấu trúc, thuật toán bên trong sản phẩm. Các ứng dụng có cấu trúc hệ thống càng lớn càng cần nhiều thời gian để kiểm tra đầy đủ.
- Kiểm thử diễn ra khi sản phẩm chưa được hoàn thiện nên các công cụ phục vụ cho mọi loại triển khai/nền tảng thường không sẵn có.
- Kiểm tra hộp trắng được đánh giá khá phức tạp và tốn kém. Developers không kiểm tra hộp trắng chi tiết có thể dẫn đến lỗi sản xuất.

Các kỹ thuật Kiểm thử hộp trắng



Các cách Kiểm thử hộp trắng

- Kiểm thử dựa trên luồng điều khiển – control flow.
- Kiểm thử dựa trên luồng dữ liệu – data flow.



KIỂM THỬ LUỒNG ĐIỀU KHIỂN

Kiểm thử luồng điều khiển

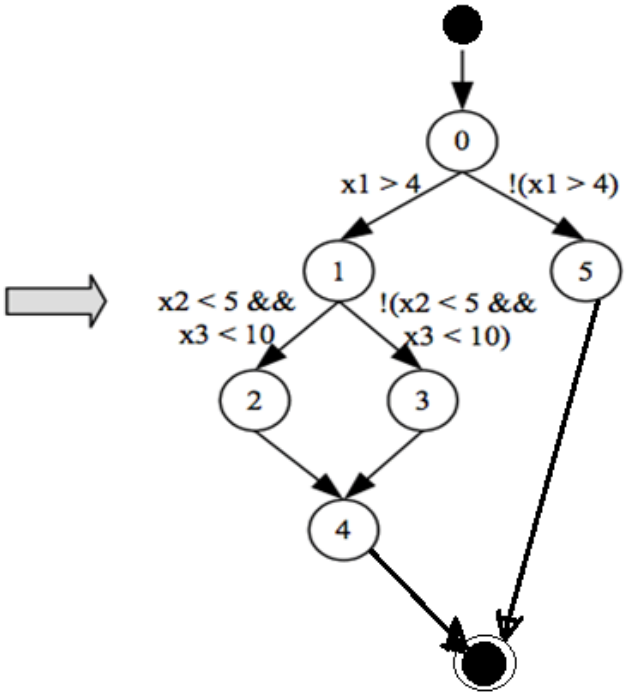
- Kiểm thử luồng điều khiển là phương pháp kiểm thử tập trung vào tính đúng của thuật toán cài đặt trong chương trình.
- Phương pháp này được xây dựng dựa trên **đồ thị luồng điều khiển**.
- Đồ thị luồng điều khiển được xây dựng thông qua mã nguồn của chương trình:
  - Là một đồ thị có hướng.
  - Các đỉnh là các câu lệnh (hoặc khối lệnh)
  - Các cạnh là dòng điều khiển giữa các câu lệnh (hoặc khối lệnh)

Kiểm thử luồng điều khiển

Program Under Test

```
void function1(int x1, int x2, int x3)
{
0   int value = 0;
    if(x1 > 4)
    {
1       if(x2 < 5 && x3 < 10)
        {
2           value = func2(x2, x3);
            cout << "func2 = " << value;
        }
        else
        {
3           value = func3(x2, x3);
            cout << "func3 =  " << value;
        }
4       return value * 2;
    }
    else
    {
5       value = func4(x1);
        return value;
    }
}
```

Control Flow Graph

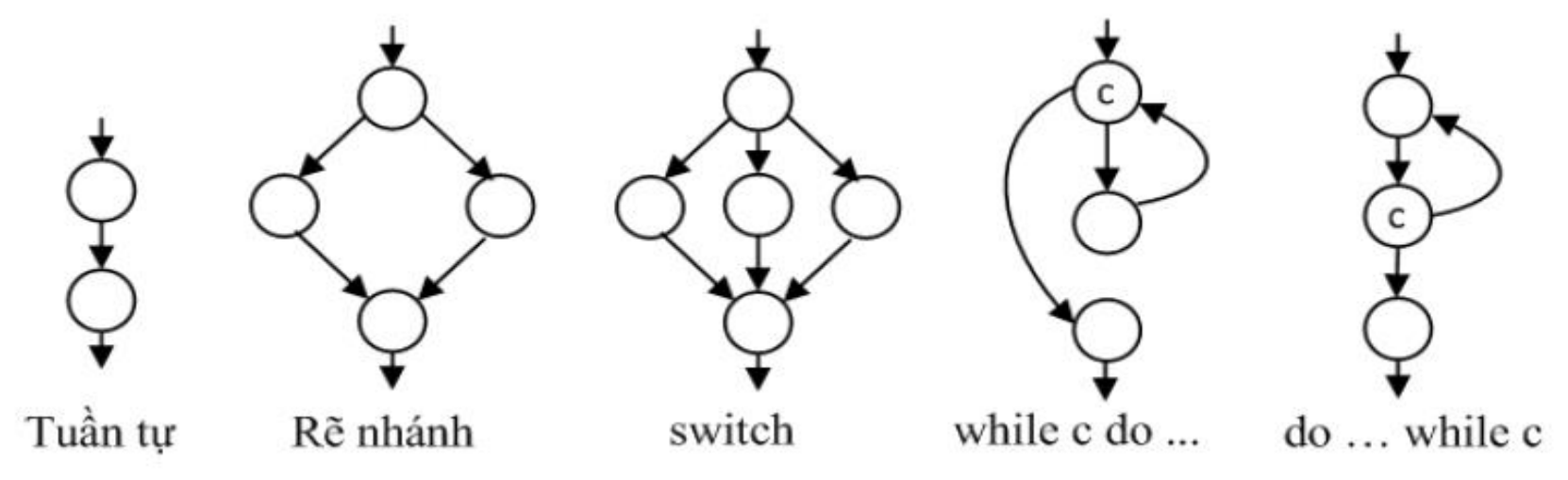


Đồ thị luồng điều khiển

❖Các nút trong đồ thị luồng điều khiển



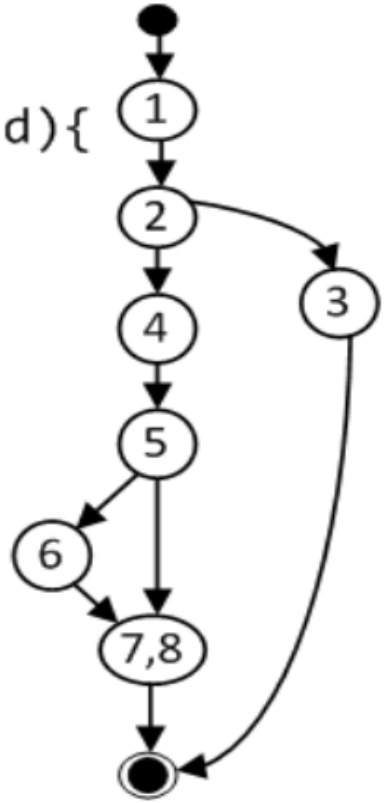
❖Các cấu trúc điều khiển



Đồ thị luồng điều khiển

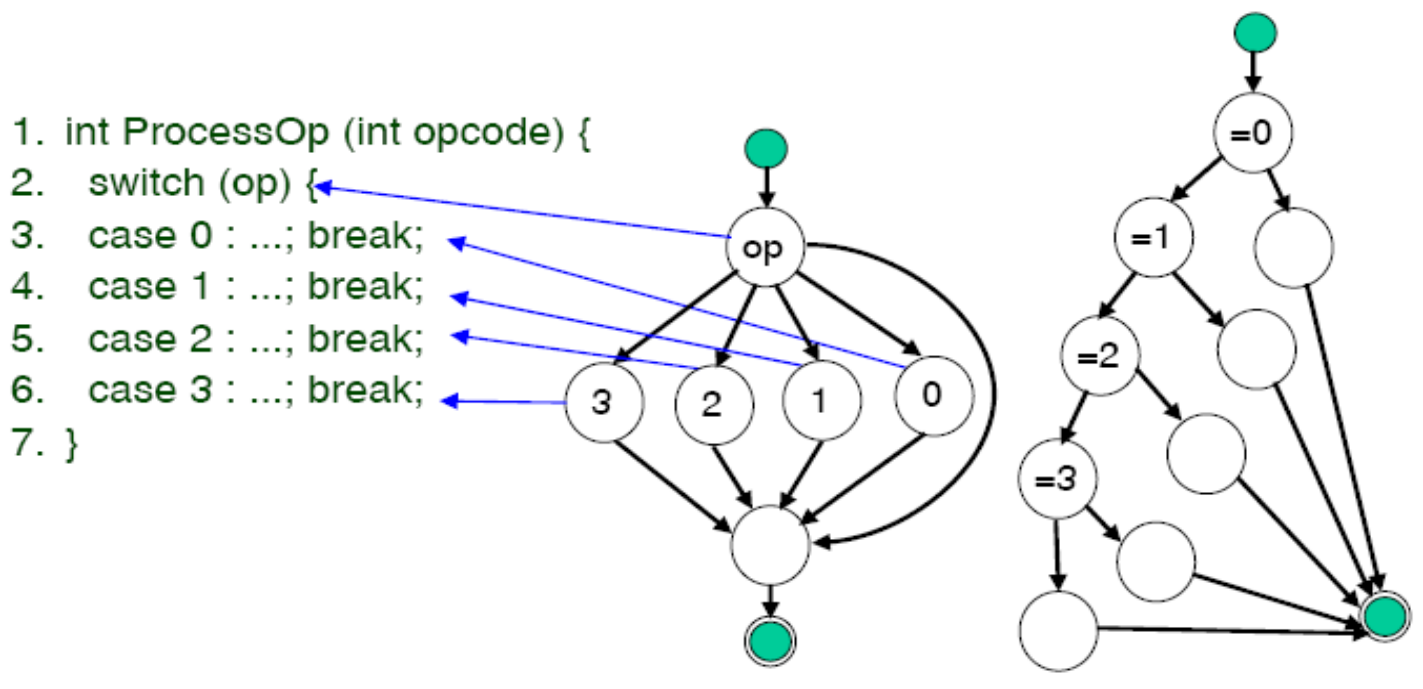
❖Ví dụ xây dựng đồ thị luồng điều khiển

```
float foo(int a, int b, int c, int d){
1.   float e;
2.   if (a==0)
3.      return 0;
4.   int x = 0;
5.   if ((a==b) || (c==d))
6.     x = 1;
7.   e = 1/x;
8.   return e;
}
```



Đồ thị luồng điều khiển nhị phân

- Là đồ thị mà các nút quyết định là nhị phân.
- Mọi đồ thị luồng điều khiển đều có thể chuyển về đồ thị luồng điều khiển nhị phân.



Độ bao phủ của kiểm thử

- Độ bao phủ của kiểm thử là tỉ lệ các thành phần thực sự được kiểm thử so với toàn bộ thành phần kiểm thử.
- Các thành phần có thể kiểm thử bao gồm: câu lệnh, điểm quyết định, điều kiện con …
- Người ta phân mức độ bao phủ trong kiểm thử theo luồng điều khiển gồm 4 mức:
  - **Phủ cấp 0:** kiểm thử những gì có thể, phần còn lại sẽ do người dùng thực thi và đưa ra kết quả ➔ không áp dụng cho việc xây dựng phần mềm thương mại.
  - Các mức còn lại được áp dụng tùy theo thực tế.

Độ bao phủ của kiểm thử

❖ **Phủ cấp 1:** Kiểm thử sao cho mỗi lệnh được thực thi ít nhất 1 lần.

- Cần 2 test case để đảm bảo:
  - ✓ 1. foo(0,0,0,0)
  - ✓ 2. foo(1,1,1,1)
- Không phát hiện được lỗi: chia 0 ở Line 7.

```
float foo(int a, int b, int c, int d)
1.  float e;
2.  if (a==0)
3.     return 0;
4.  int x = 0;
5.  if ((a==b) || (c==d))
6.    x = 1;
7.  e = 1/x;
8.  return e;
}
```

Độ bao phủ của kiểm thử

❖ **Phủ cấp 2:** Kiểm thử sao cho mỗi điểm quyết định đều được thực thi cả 2 trường hợp: true – false.

- Cần 3 test case để đảm bảo:
  - ✓ 1. foo(0,0,0,0)
  - ✓ 2. foo(1,1,1,1)
  - ✓ 3. foo(1,2,1,2)
- Line 2: test case 1 + 2.
- Line 5: test case 2 + 3: Phát hiện lỗi ở Line 7.

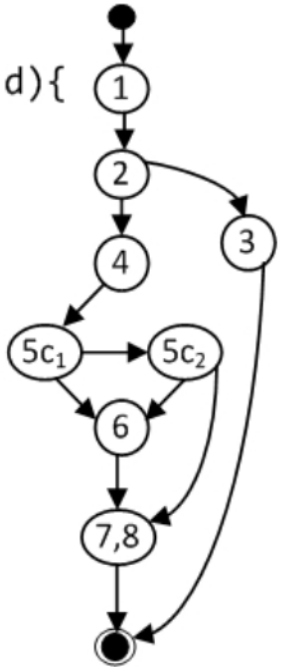
```
float foo(int a, int b, int c, int d)
1.  float e;
2.  if (a==0)
3.     return 0;
4.  int x = 0;
5.  if ((a==b) || (c==d))
6.    x = 1;
7.  e = 1/x;
8.  return e;
}
```

Độ bao phủ của kiểm thử

- **Phủ cấp 3:** Các điểm quyết định được chuyển sang đồ thị luồng điều khiển nhị phân và kiểm thử sao cho mỗi điều kiện đều được thực thi cả 2 trường hợp: true – false.

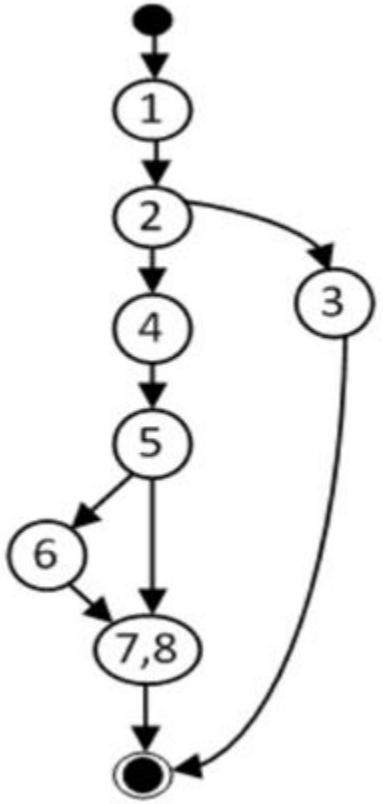
Predicate	True	False
a ==0	TC 1 : foo(0, 0, 0, 0) return 0	TC 2 : foo(1, 1, 1, 1) return 1
(a==b)	TC 2 : foo(1, 1, 1, 1) return 1	TC 3 : foo(1, 2, 1, 2) division by zero!
(c==d)	TC 4 : foo(1, 2, 1, 1) Return 1	TC 3 : foo(1, 2, 1, 2) division by zero!

```
float foo(int a, int b, int c, int d){
1.  float e;
2.  if (a==0)
3.    return 0;
4.  int x = 0;
5.  if ((a==b) || (c==d))
          5c1      5c2
6.    x = 1;
7.  e = 1/x;
8.  return e;
}
```



Đường thi hành

- Đường thi hành – Execution path – là một test case cần được xây dựng để kiểm thử cho một chương trình (hàm, module) của phần mềm.
- Trong đồ thị luồng điều khiển, đường thi hành là đường đi từ điểm bắt đầu đến điểm kết thúc.
- Ví dụ: Đồ thị trên có 1 đường thi hành là
  - Start->1->2->3->End
- Một đồ thị có thể có nhiều đường thi hành



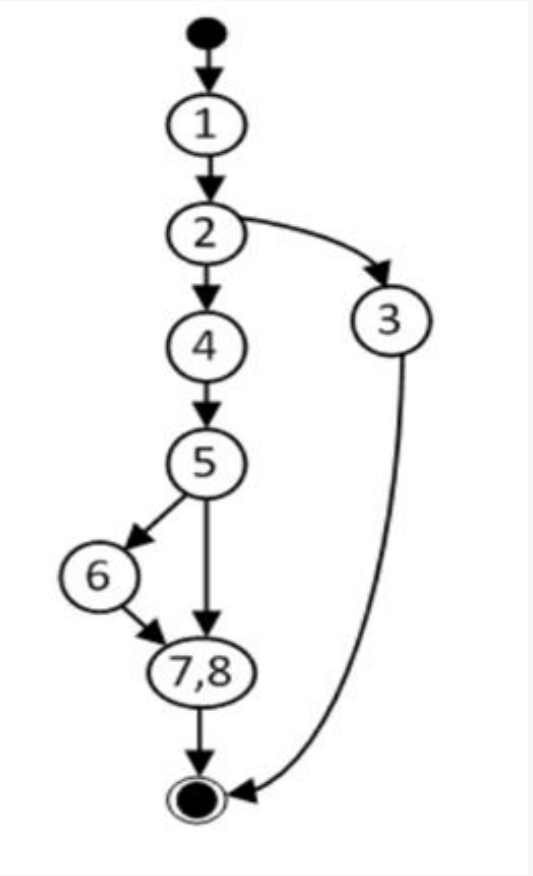
Quy trình xác định đường thi hành tuyến tính độc lập

- Xác định đường đầu tiên thông qua nhánh bên trái của các nút quyết định – đường cơ bản.
- Dựa trên đường cơ bản, xác định các đường tiếp theo thông qua lần lượt nhánh phải của mỗi nút quyết định. Giữ nguyên các phần còn lại tại các nút quyết định.
- Tiếp tục thay đổi cho đến khi duyệt hết các nút quyết định của đồ thị luồng điều khiển.
- Số đường tuyến tính độc lập được xác định dựa trên độ phức tạp của đồ thị.

Quy trình xác định đường thi hành tuyến tính độc lập

❖ **Ví dụ:** Đồ thị bên có các đường thi hành tuyến tính độc lập sau:

- Start->1->2->4->5->6->7,8->End
- Start->1->2->4->5->7,8->End
- Start->1->2->3->End



Độ phức tạp của đồ thị

- Độ phức tạp – Cyclomatic [C] – của đồ thị được xác định

$$C(Vg) = E - N + 2$$

E: số cạnh, N: số đỉnh

- Trong đồ thị luồng điều khiển nhị phân, độ phức tạp của đồ thị được xác định thông qua

$$C(Vg) = P + 1$$

P: số nút quyết định

- Độ phức tạp C chính là số đường thi hành tuyến tính độc lập cần có để đảm bảo kiểm thử đầy đủ – đảm bảo phủ cấp 3.

Quy trình kiểm thử dựa trên luồng điều khiển

- **B1:** Từ thành phần kiểm thử, xây dựng đồ thị luồng điều khiển tương ứng – luồng điều khiển nhị phân.
- **B2:** Xác định độ phức tạp C(Vg) của đồ thị.
- **B3:** Xây dựng C đường thi hành tuyến tính độc lập dựa trên đồ thị luồng điều khiển.
- **B4:** Với mỗi đường thi hành tuyến tính, xây dựng một test case tương ứng: xác định các giá trị để chương trình chạy theo đường thi hành đó.
- **B5:** Kiểm thử các test case được tạo ra, so sánh với kết quả mong muốn và lập ma trận bao phủ cho thành phần kiểm thử.

Ví dụ

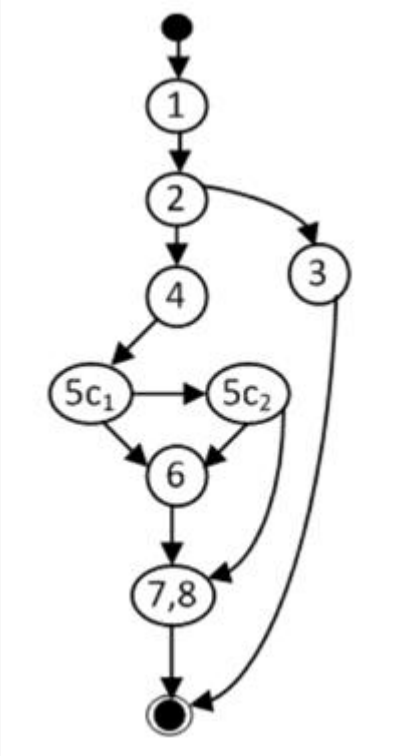
- Xây dựng các ca kiểm thử cho hàm sau để đảm bảo phủ cấp 3
  - Hàm bên tính giá trị e= 1/[(a-b)*(c-d)]
  - Nếu a=0 thì e=0
  - Nếu a=b hoặc c=d thì e=1

```
float foo(int a, int b, int c, int d)
1.  float e;
2.  if (a==0)
3.     return 0;
4.  int x = 0;
5.  if ((a==b) || (c==d))
6.    x = 1;
7.  e = 1/x;
8.  return e;
}
```

Quy trình xác định đường thi hành tuyến tính độc lập

❖ Ví dụ:

- Độ phức tạp của đồ thị:
  C(Vg)=P+1=3+1=4
- Các đường thi hành tuyến tính độc lập
  1. Start -> 1 -> 2 -> 4 -> $5c_1$ -> 6 -> 7,8 -> End
  2. Start -> 1 -> 2 -> 4 -> $5c_1$ -> $5c_2$ -> 6 -> 7,8 -> End
  3. Start -> 1 -> 2 -> 4 -> $5c_1$ -> $5c_2$ -> 7,8 -> End
  4. Start -> 1 -> 2 -> 3 -> End

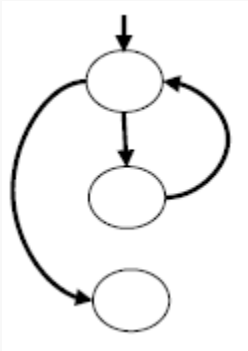


Xác định Test case tương ứng và thực hiện kiểm thử

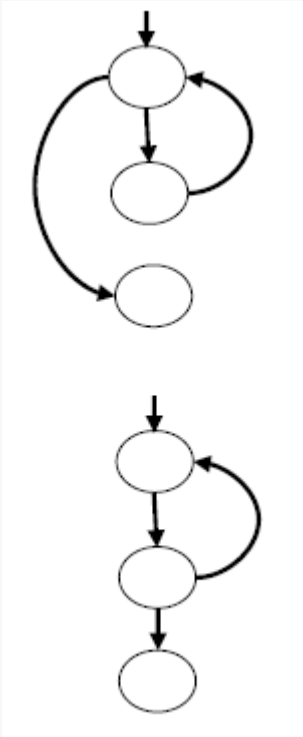
ID	Test Path	Inputs	Expected Outputs	RO	Note
tc1	1; 2; 4; 5c1; 6; 7,8	**2,2,1,2**	**1**	**Pass**	
tc2	1; 2; 4; 5c1; 5c2; 6; 7,8	**1,2,3,3**	1	**Pass**	
tc3	1; 2; 4; 5c1; 5c2; 7,8	**2, 3, 3,1**	**-0.5**	**Fail**	**Lỗi chia 0**
tc4	1; 2; 3	**0,1,2,2**	**0**	**Pass**	

Kiểm thử vòng lặp

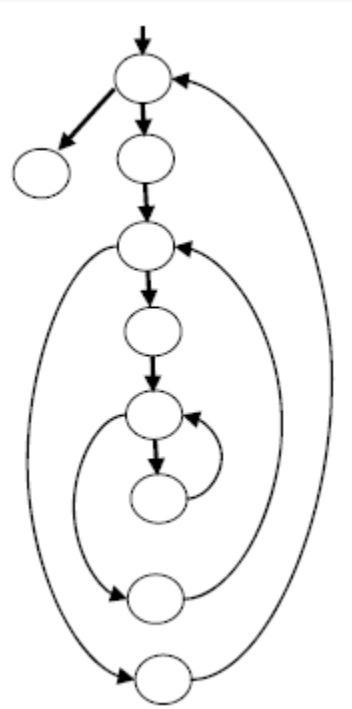
- Các loại vòng lặp:
  - Lặp đơn
  - Lặp đơn liền kề nhau
  - Lặp lồng nhau



Lặp đơn



Lặp đơn liền kề



Lặp đơn lồng nhau

Kiểm thử vòng lặp

- Nguyên tắc với lặp đơn bao gồm 7 test case:
  - Lặp 0 bước
  - Lặp 1 bước
  - Lặp 2 bước
  - Lặp k bước với 2 < k < n-1
  - Lặp n-1 bước
  - Lặp n bước
  - Lặp n+1 bước
- Nguyên tắc với lặp đơn liền kề nhau: thực hiện 7 test case với mỗi lặp đơn đồng thời cho mỗi vòng lặp.

Kiểm thử vòng lặp

- Nguyên tắc với lặp lồng nhau:
  - Kiểm thử vòng lặp trong cùng: cho các vòng ngoài chạy với giá trị mặc định – kiểm tra vòng lặp trong cùng thông qua 7 test case.
  - Kiểm thử từng vòng lặp tiếp theo: giữ các vòng lặp bên trong và bên ngoài với các giá trị mặc định, thực thi vòng lặp kiểm thử với 7 test case tương ứng.

Kiểm thử vòng lặp

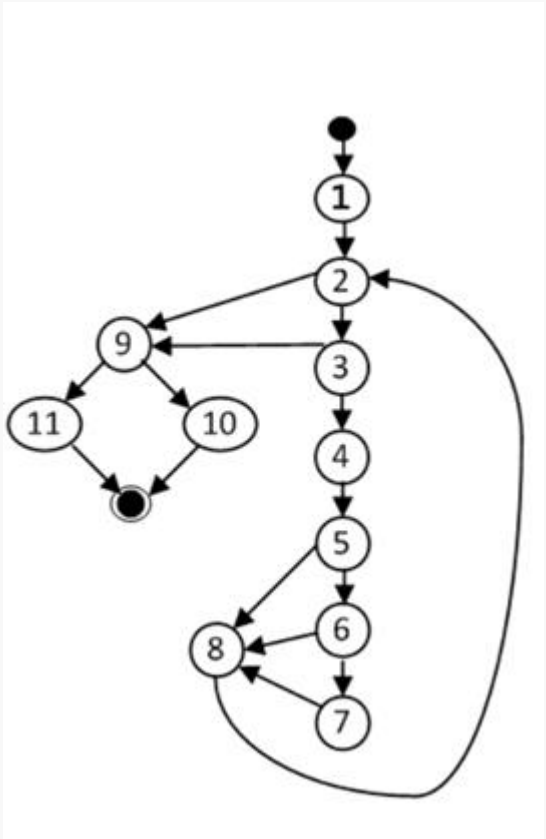
❖ **Chú ý:**

- Trong một số trường hợp chúng ta có thể không xác địnhđược số lần lặp tối đa của các vòng lặp. → Trong trường hợp này, chúng ta chỉ cần sinh bốn ca kiểm thử đầu tiên.
- Tương tự, trong một số các trường hợp khác, chúng ta không thể sinh ca kiểm thử để vòng lặp thực hiện n + 1 lần (trường hợp thứ 7) → Khi đó, chúng ta chỉ cần sinh sáu ca kiểm thử còn lại (các trường hợp từ 1–6).

Ví dụ kiểm thử vòng lặp

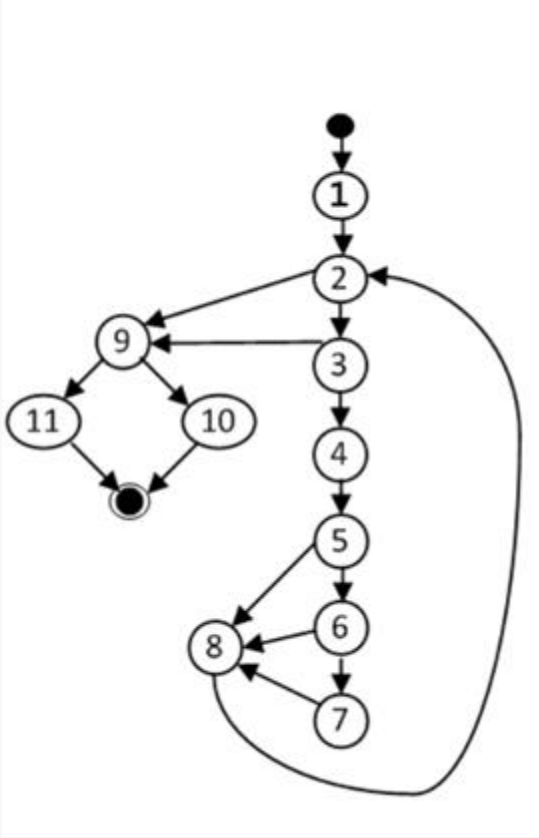
- Thiết kế các Test case cần có cho hàm sau để đạt độ phủ cấp 3:

```
double average(double value[], double min, double
      max, int& tcnt, int& vcnt) {
      double sum = 0;
      int i = 1;                    |1
      tcnt = vcnt = 0;              |
      while (value[i] <> -999 && tcnt <100) {
                  2                    3
            tcnt++; |4
            if (min<=value[i] && value[i]<= max){
                        5                6
                        sum += value[i];|7
                        vcnt ++;
            }
            i++;|8
      }//end while
      if (vcnt > 0)
            9
            return sum/vcnt; |10
      return -999; |11
}//end
```



Ví dụ kiểm thử vòng lặp

- **B1:** Xây dựng đồ thị luồng điều khiển nhị phân
- **B2:** Tính độ phức tạp:
  - C(Vg)=P+1=5+1=6
- **B3:** Xác định các đường thi hành
  1. Start→1→2→9→11→End
  2. Start→1→2→9→10 →End
  3. Start→ 1→2→3→9→10 →End
  4. Start→ 1→2→3→4→5→8→2→9→11 →End
  5. Start→ 1→2→3→4→5→6→8→2→9→11 →End
  6. Start→ 1→2→3→4→5→6→7→8→2→9→10 →End



Ví dụ kiểm thử vòng lặp

❖ **B4:** Xác định Test case

ID	Test Path	Inputs	Expected Outputs	Result Output	Note
TC1	1→2→9→11	[-999,1,2],1,2	-999		
TC2	1→2→9→10				TC6
TC3	1→2→3→9→10				TC6
TC4	1→2→3→4→5→8→2→9→11	[0,-999],1,2	-999		
TC5	1→2→3→4→5→6→8→2→9→11	[3,-999],1,2	-999		
TC6	1→2→3→4→5→6→7→8→2→9→10	[1,-999],1,2	1		
TCL0	Lặp 0 lần:	[-999,1,2	-999		TC1
TCL1	Lặp 1 lần:	[1,-999],1,2	1		TC6
TCL2	Lặp 2 lần	[1,2,-999],1,2	1.5		
TCLk	Lặp k=5 lần	[1,2,3,4,5,-999],1,5	3		
TCLn-1	Lặp 99 lần	[1,2…,99,-999],1,99	50		
TCLn	Lặp 100 lần	[1,2…,100,-999],1,100	50.5		

KIỂM THỬ LUỒNG DỮ LIỆU

Khái niệm

- Mục tiêu của chương trình là việc xử lý dữ liệu. Dữ liệu được biểu diễn trong chương trình thông qua **biến**.

→Bản chất của phương pháp kiểm thử luồng dữ liệu là việc kiểm thử chu trình sống của mỗi biến trong chương trình, trong module hay trong mỗi hàm của chương trình.

- Đây là phương pháp kiểm thử chặt chẽ nhằm xác định được việc dùng không hợp lý của mỗi biến:
  - Khai báo và nhập dữ liệu không đúng: dư thừa, không hợp lệ…
  - Thiếu định nghĩa, khởi tạo trước khi dùng.
  - Biến định nghĩa nhưng không được dùng.
  - Etc.

Khái niệm

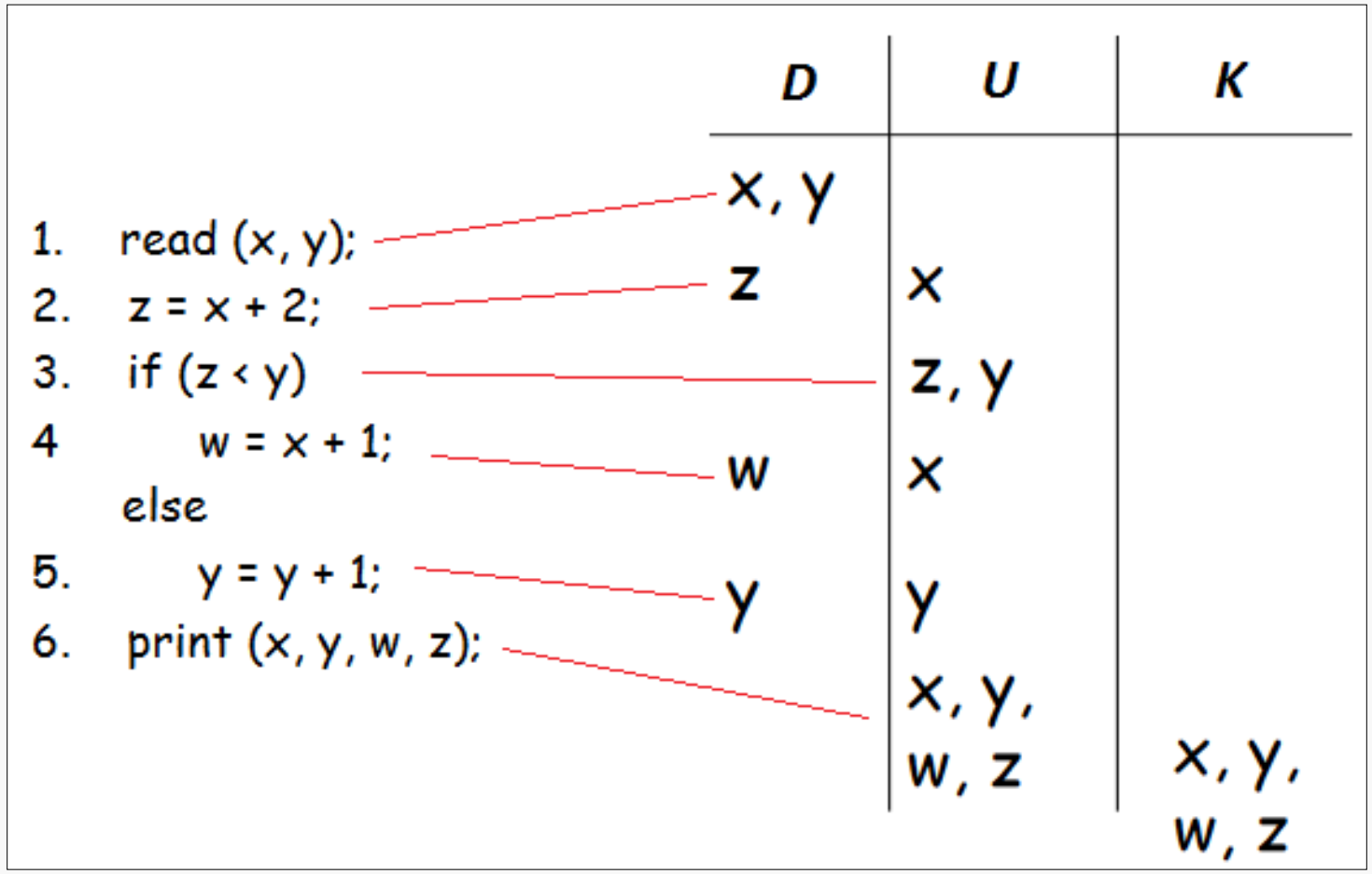
- Phương pháp kiểm thử luồng dữ liệu được xây dựng dựa trên đồ thị luồng điều khiển mà tại đó, thay vì kiểm thử luồng thì người ta kiểm tra các trạng thái hoạt động của biến.
- Mỗi biến bao gồm có 3 trạng thái:
  - Khởi tạo (Definition – d)
  - Sử dụng (Use – u)
  - Kết thúc (Kill – k)

Các trạng thái của biến

- **Khởi tạo – Definition [d]**
  - Khai báo một biến
    Example int a;
  - Gán giá trị mới cho biến
    Example a = 5; a = b + 1;
- **Sử dụng – Use [u]**
  - Sử dụng trong phép điều kiện (Pu)
    Example if (a>0) ➔ a: Pu
  - Sử dụng trong phép tính, trong lời gọi hàm (Cu)
    Example: a = b + 1 ➔ b: Cu
- **Kết thúc – Kill [k]**

Các trạng thái của biến

❖ Ví dụ



Phân tích cặp trạng thái của biến

- Các cặp trạng thái của biến cho ta xác định được việc hoạt động của chương trình có phù hợp hay không.
- Với 3 trạng thái chúng ta có 9 cặp:
  1. dd: biến chưa dùng lại được định nghĩa lại ➔ KO – có khả năng dư thừa.

     Example: int x; x = 5; ➔ int x = 5;
  2. du: biến được định nghĩa và được dùng ➔ OK.
  3. dk: biến được định nghĩa rồi giải phóng ➔ KO – có khả năng dư thừa.
  4. ud: biến được dùng rồi lại được khởi tạo lại ➔ OK
  5. uu: biến được dùng rồi lại được dùng tiếp ➔ OK
  6. uk: biến được dùng rồi được giải phóng ➔ OK

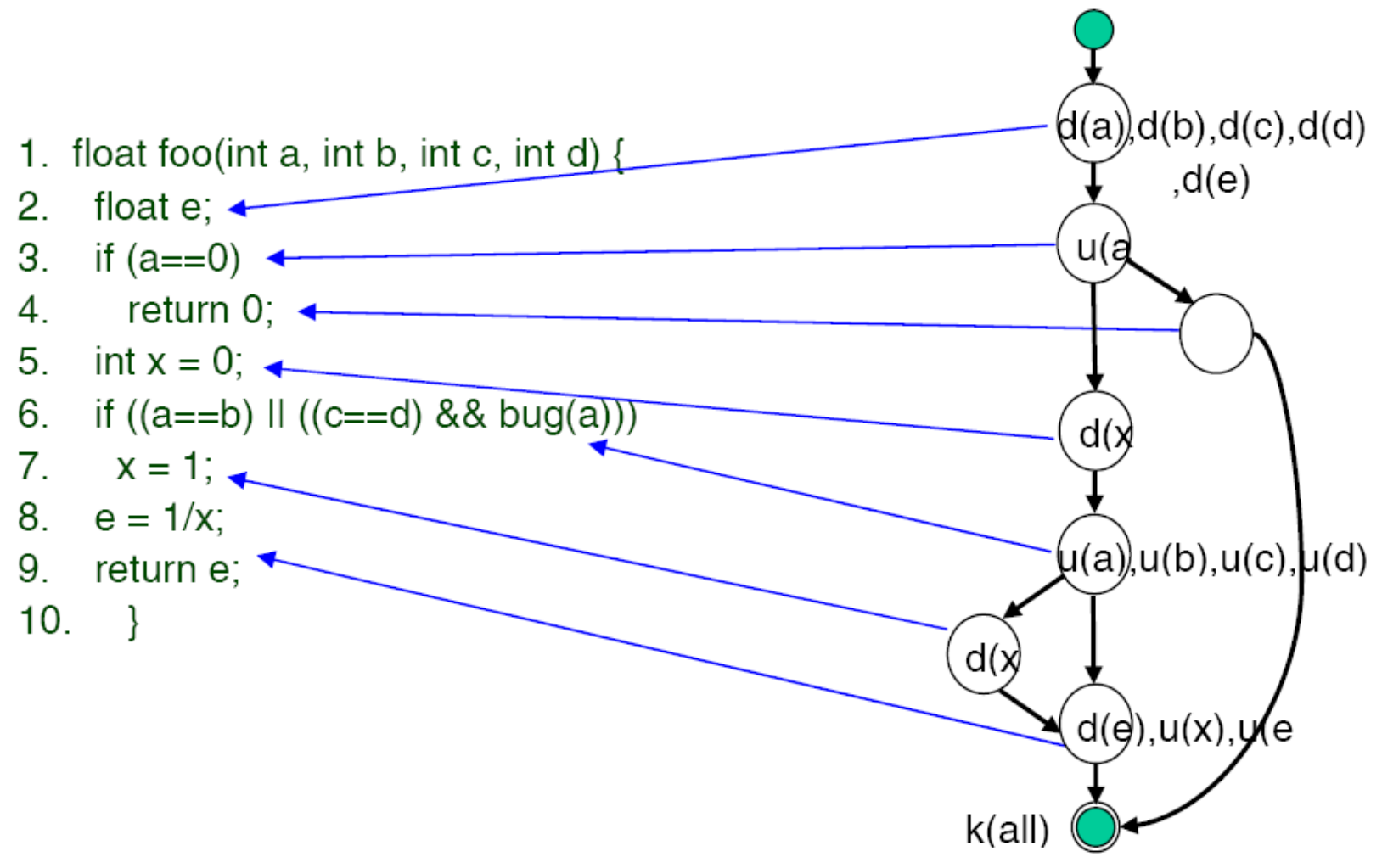
Phân tích cặp trạng thái của biến

- Với 3 trạng thái chúng ta có 9 cặp:
  7. kd: biến được giải phóng rồi khởi tạo lại ➔ OK
  8. ku: biến được giải phóng rồi lại được dùng tiếp ➔ KO – chắc chắn là lỗi.
  9. kk: biến được giải phóng rồi lại được giải phóng ➔ KO – dư thừa lệnh.
- Các trạng thái độc lập
  1. u, k: biến sử dụng chưa khai báo ➔ compile error.
  2. d: biến khai báo nhưng không sử dụng ➔ dư thừa.

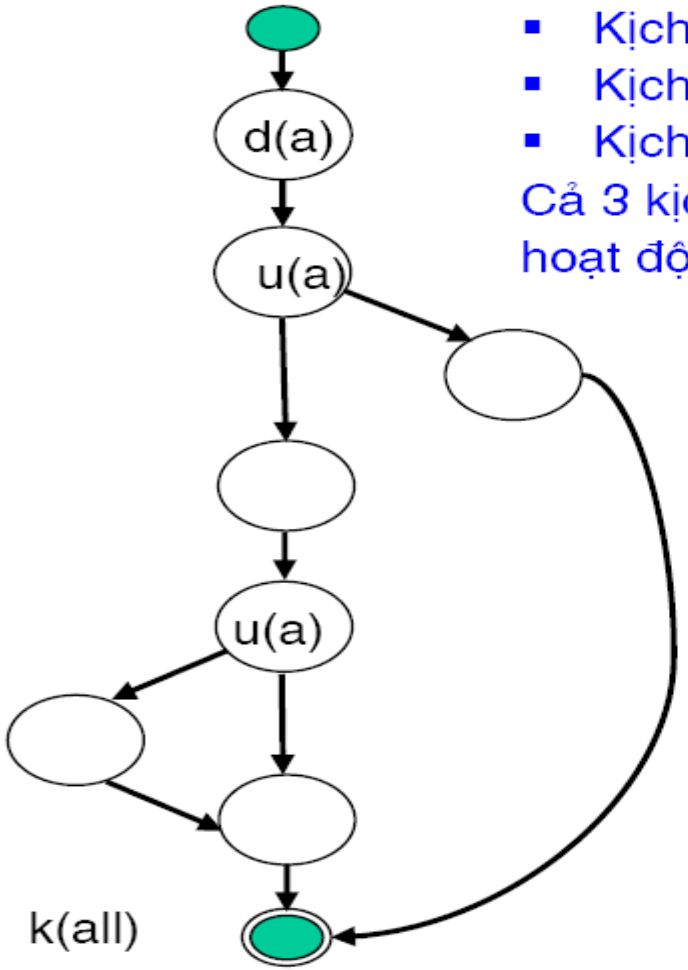
Quy trình kiểm thử luồng dữ liệu

- **B1:** Từ thành phần kiểm thử, xây dựng đồ thị luồng điều khiển tương ứng – luồng điều khiển nhị phân.
- **B2:** Trên đồ thị luồng điều khiển, xây dựng đồ thì luồng dữ liệu.
- **B3:** Xác định độ phức tạp C(Vg) của đồ thị.
- **B4:** Với mỗi một biến trong chương trình:
  - Xây dựng tối đa C đường thi hành tuyến tính độc lập dựa trên đồ thị luồng dữ liệu.
  - Với mỗi đường thi hành tuyến tính, xác định các bất thường của trạng thái biến.
  - Với mỗi trạng thái bất thường ➔ lập báo cáo để đánh giá (bug?)

Ví dụ


1. float foo(int a, int b, int c, int d) {
2. float e;
3. if (a==0)
4. return 0;
5. int x = 0;
6. if ((a==b) || ((c==d) && bug(a)))
7. x = 1;
8. e = 1/x;
9. return e;
10. }
d(a),d(b),d(c),d(d)
,d(e)
u(a
d(x
u(a),u(b),u(c),u(d)
d(x
d(e),u(x),u(e
k(all)

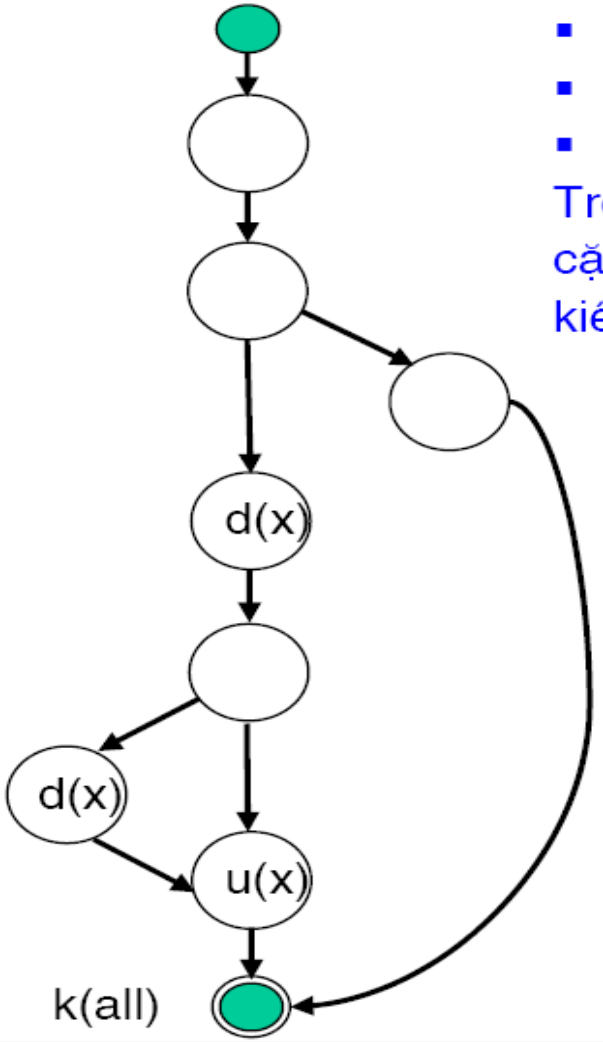
Đồ thị luồng dữ liệu biến a



- Kịch bản 1 : ~duuk
- Kịch bản 2 : ~duuk (giống kịch bản 1).
- Kịch bản 3 : ~duk

Cả 3 kịch bản trên đều không chứa cặp đôi hoạt động nào bất thường cả.

Đồ thị luồng dữ liệu biến x



- Kịch bản 1 : ~dduk
- Kịch bản 2 : ~duk
- Kịch bản 3 : ~

Trong 3 kịch bản trên, chỉ có kịch bản 1 có chứa cặp đôi dd bất thường nên cần tập trung chú ý kiểm tra xem có phải là lỗi không.

Trân trọng cảm ơn!